

1. Lineární model báзовých funkcí

Bázové funkce a jejich příklady

- **Bázové funkce:**
 - Funkce, které nahrazují původní příznaky jejich transformovanými variantami.
 - Uvažujeme $M \in \mathbb{N}$ funkcí $\varphi_1 \dots \varphi_M$, z $\mathcal{X} \rightarrow \mathbb{R}$, kde $\mathcal{X}$ je prostor všech možných hodnot vektoru příznaku $\mathbf{X} = (X_1, \dots, X_p)^T$. Typicky $\mathcal{X} = \mathbb{R}^p$ nebo alespoň $\mathcal{X} \subset \mathbb{R}^p$.
- **Příklady:**
 - $\varphi(\mathbf{x}) = x_i$ – identita (většinou prvních p funkcí jsou identity, abychom si ponechali původní příznaky).
 - $\varphi(\mathbf{x}) = x_i^2$, $\varphi(\mathbf{x}) = x_k x_l$ – mocniny příznaků a jejich různé součiny (polynomiální regrese).
 - $\varphi(\mathbf{x}) = \log(x_i)$, $\sin(x_i)$, $\sqrt{x_i}$ atd. – nelineární transformace.
 - $\varphi(\mathbf{x}) = \mathbb{1}_{(a,b)}(x_i)$, kde $\mathbb{1}_A(x_i) = 1$, když $x_i \in A$ a $\mathbb{1}_A(x_i) = 0$, když $x_i \notin A$ – indikátory množin. Můžu rozdělit prostor na kousky (množiny).
 - $\varphi(\mathbf{x}) = h(\|\mathbf{x} - \mathbf{x}_i\|)$, kde $\mathbf{x}_i$ je i -tý bod trénovací množiny a h je nějaká funkce – tzv. **radiální básová funkce** centrovaná v bodech trénovací množiny.

Lineární model báзовých funkcí

- **Definice:**
 - Vysvětlovaná proměnná Y v bodě $\mathbf{x} = (x_1, \dots, x_p)^T$ hodnot příznaků $\mathbf{X}$ je určena vztahem:

$$Y = w_1 \varphi_1(\mathbf{x}) + \dots + w_M \varphi_M(\mathbf{x}) + \varepsilon = \mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}) + \varepsilon,$$

kde $\boldsymbol{\varphi} : \mathcal{X} \rightarrow \mathbb{R}^M$ je vektorová funkce definovaná jako $\boldsymbol{\varphi} = (\varphi_1(\mathbf{x}), \dots, \varphi_M(\mathbf{x}))^T$ pro všechna $\mathbf{x} \in \mathcal{X}$, $\mathbf{w} = (w_1, \dots, w_M)^T$ je vektor neznámých parametrů, a ε je náhodná veličina se střední hodnotou $\mathbb{E}\varepsilon = 0$.

- Prostě vezmu normální lineární model, ale místo $\mathbf{w}^T \mathbf{x}$ mám $\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x})$.
- Model pro trénovací množinu tvořenou N dvojicemi $(Y_1, x_1), \dots, (Y_N, x_N)$ můžeme ve vektorovém tvaru zapsat jako:

$$\mathbf{Y} = \boldsymbol{\Phi} \mathbf{w} + \boldsymbol{\varepsilon},$$

kde

$$\boldsymbol{\Phi} = \begin{pmatrix} \boldsymbol{\varphi}(\mathbf{x}_1)^T \\ \vdots \\ \boldsymbol{\varphi}(\mathbf{x}_N)^T \end{pmatrix} = \begin{pmatrix} \varphi_1(\mathbf{x}_1) & \cdots & \varphi_M(\mathbf{x}_1) \\ \vdots & \ddots & \vdots \\ \varphi_1(\mathbf{x}_N) & \cdots & \varphi_M(\mathbf{x}_N) \end{pmatrix}, \quad \mathbf{Y} = \begin{pmatrix} Y_1 \\ \vdots \\ Y_N \end{pmatrix}, \quad \boldsymbol{\varepsilon} = \begin{pmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_N \end{pmatrix}.$$

Explicitní řešení optimalizační úlohy trénování

- Podobně jako u standardního lineárního modelu budeme minimalizovat $\text{RSS}_\lambda(\mathbf{w})$. V našem případě ale s transformovanými příznaky, tedy:

$$\text{RSS}_\lambda(\mathbf{w}) = \|\mathbf{Y} - \Phi\mathbf{w}\|^2 + \lambda\mathbf{w}^T\mathbf{w}$$

To vede na **normální rovnici**:

$$\Phi^T\mathbf{Y} - \Phi^T\Phi\mathbf{w} - \lambda\mathbf{w} = \mathbf{0},$$

kde $\mathbf{0}$ je nulový vektor s M složkami.

Pro $\lambda > 0$ existuje jednoznačné řešení:

$$\mathbf{w}_\lambda = (\Phi^T\Phi + \lambda\mathbf{I})^{-1}\Phi^T\mathbf{Y}.$$

Predikce hodnoty Y v bodě $\mathbf{x}$ je potom:

$$\hat{Y} = \mathbf{w}_\lambda^T \boldsymbol{\varphi}(\mathbf{x})$$

2. Jádrová regrese

Přechod od lineárního modelu báзовých funkcí k jádrové regresi (jádrový trik)

- Duální reprezentace:**
 - Začneme s lineárním modelem báзовých funkcí:

$$\text{RSS}_\lambda(\mathbf{w}) = \|\mathbf{Y} - \Phi\mathbf{w}\|^2 + \lambda\mathbf{w}^T\mathbf{w}$$

- Ted' budeme uvažovat hodnoty $\mathbf{w}$ ve tvaru:

$$\mathbf{w} = \Phi^T \boldsymbol{\alpha} \in \mathbb{R}^N,$$

tj. omezíme $\mathbf{w}$ na podprostor $\mathbb{R}^M$ generovaný vektory $\boldsymbol{\varphi}(\mathbf{x}_1), \dots, \boldsymbol{\varphi}(\mathbf{x}_N)$.

- Dosazením do $\text{RSS}_\lambda(\mathbf{w})$ dostáváme:

$$\text{RSS}_\lambda(\boldsymbol{\alpha}) = \|\mathbf{Y} - \Phi\Phi^T\boldsymbol{\alpha}\|^2 + \lambda\boldsymbol{\alpha}^T\Phi\Phi^T\boldsymbol{\alpha}$$

- Věta:**

Pro $\lambda > 0$ platí:

$$\arg\min_{\boldsymbol{\alpha}} \text{RSS}_\lambda(\boldsymbol{\alpha}) = \arg\min_{\mathbf{w}} \text{RSS}_\lambda(\mathbf{w})$$

Navíc, pokud $\mathbf{w}$ minimalizuje $\text{RSS}_\lambda(\mathbf{w})$, tak $\boldsymbol{\alpha} = \mathbf{w}^T \Phi$ minimalizuje $\text{RSS}_\lambda(\boldsymbol{\alpha})$.

Na druhou stranu, pokud $\boldsymbol{\alpha}$ minimalizuje $\text{RSS}_\lambda(\boldsymbol{\alpha})$, tak $\mathbf{w} = \Phi^T \boldsymbol{\alpha}$ minimalizuje $\text{RSS}_\lambda(\mathbf{w})$.

- Důkaz:**

Není potřeba ke zkoušce.

- Jádrový trik:**

- Vytvoříme ekvivalentní duální reprezentaci celého modelu (viz níže).

- V této reprezentaci se body $\mathbf{x}, \mathbf{x}_1, \dots$ vyskytují pouze ve tvaru skalárních součinů jejich transformací pomocí báзовých funkcí, které můžeme kompletně vyjádřit pomocí **jádrové funkce** $k(\mathbf{x}, \cdot)$.
- Nahrazení skalárních součinů jádrovou funkcí se nazývá **jádrový trik**.
- Rovnou také můžeme začít jádrovou funkcí bez explicitního zadávání báзовých funkcí.
- Můžeme pak pracovat v **příznakových prostorech velké** (až nekonečné) **dimenze**.
- **Maticová inverze** má časovou složitost (M) resp. (N) pro odhad $\mathbf{w}$ resp. .

Definice jádrové funkce a příklady

- **Definice:**

Funkci $k : \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}$, definovanou jako:

$$k(\mathbf{x}, \cdot) = \boldsymbol{\varphi}(\mathbf{x})^T \boldsymbol{\varphi}(\cdot) \quad \text{obzvláště } \mathbf{x}, \cdot \in \mathbb{R}^p.$$

- **Příklady:**

- **Polynomiální jádro:**

$$k(\mathbf{x}, \cdot) = (\mathbf{x}^T + 1)$$

- **Lineární jádro:**

$$k(\mathbf{x}, \cdot) = \mathbf{x}^T$$

- **Gaussovské jádro:**

$$k(\mathbf{x}, \cdot) = -\|\mathbf{x} - \cdot\|^2$$

Gramova matice a explicitní řešení optimalizační úlohy trénování

- **Definice:**

Gramovu matici (angl. Gram Matrix) definujeme jako:

$$\mathbf{G} = \boldsymbol{\Phi} \boldsymbol{\Phi}^T \in \mathbb{R}^{N, N}$$

- **Pozorování:**

- Gramova matice je **symetrická**.
- Gramova matice je **pozitivně semidefinitní**:

$$\mathbf{G}^T = \boldsymbol{\Phi}^T \boldsymbol{\Phi} = (\boldsymbol{\Phi}^T)^T (\boldsymbol{\Phi}^T) = \|\boldsymbol{\Phi}^T\|^2 \geq 0$$

pro každé $\mathbf{x} \in \mathbb{R}^N$.

- $\text{RSS}_\lambda()$ můžeme pomocí Gramovy matice vyjádřit jako:

$$\text{RSS}_\lambda() = \|\mathbf{Y} - \boldsymbol{\Phi} \boldsymbol{\beta}\|^2 + \lambda^T \boldsymbol{\beta}$$

- Gramova matice je **plně určena jádrovou funkcí**:

$$K_{ii} = (\Phi \Phi^T)_{ii} = \sum_{l=1}^M \varphi_l(\mathbf{x}_i) \varphi_l(\mathbf{x}_i) = k(\mathbf{x}_i, \mathbf{x}_i)$$

- **Řešení optimalizační úlohy**:

- Předpokládejme, že jsme našli minimalizující $\mathbf{w}$ v $\text{RSS}_\lambda(\mathbf{w})$.
- Odpovídající $\mathbf{w} = \Phi^T$ minimalizuje $\text{RSS}_\lambda(\mathbf{w})$.
- Pro predikci Y v bodě $\mathbf{x}$ máme:

$$Y = \mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}) = \sum_{l=1}^M w_l \varphi_l(\mathbf{x}) = \sum_{l=1}^M w_l k(\mathbf{x}_l, \mathbf{x})$$

- Pro gradient $\text{RSS}_\lambda(\mathbf{w})$ (derivujeme podle $\mathbf{w}$) platí:

$$\nabla \text{RSS}_\lambda(\mathbf{w}) = 2(\mathbf{Y} - \Phi \mathbf{w}) + 2\lambda \mathbf{w}$$

$$\text{RSS}_\lambda(\mathbf{w}) = -2(\mathbf{Y} - \Phi \mathbf{w}) + 2\lambda \mathbf{w}.$$

- Položíme rovno nule (nulovému vektoru):

$$(\mathbf{Y} - \Phi \mathbf{w} - \lambda \mathbf{w}) = \mathbf{0}$$

- Protože je Gramova matice **pozitivně semidefinitní**, je $\Phi \Phi^T + \lambda \mathbf{I}$ regulární a dostáváme:

$$\mathbf{w} = (\Phi \Phi^T + \lambda \mathbf{I})^{-1} \Phi^T \mathbf{Y}$$

- Tedy i optimální $\mathbf{w}$ můžeme spočítat pomocí Gramovy matice a tedy jádrové funkce.

Predikce v jádrové regresi

- **Predikce** (pomocí jádrové funkce):

$$Y = \sum_{i=1}^N k(\mathbf{x}_i, \mathbf{x})$$

Jádrové modely:

- Uvažujme, že data pocházejí ze skutečného modelu:

$$Y = f(\mathbf{x}) + \varepsilon,$$

kde f je nějaká neznámá funkce a ε je náhodná veličina s $\mathbb{E}\varepsilon = 0$.

- V **lineárním modelu básových funkcí** hledáme f ve tvaru:

$$f(\mathbf{x}) = w_1 \varphi_1(\mathbf{x}) + \dots + w_M \varphi_M(\mathbf{x}).$$

- V obecném **jádrovém modelu** s jádrovou funkcí k máme f ve tvaru:

$$f(\mathbf{x}) = \sum_{i=1}^M k(\mathbf{x}_i, \mathbf{x}),$$

kde $\mathbf{x}_1, \dots, \mathbf{x}_M \in \mathcal{X}$ jsou nějaké středové body.

- Speciálním případem jádrových modelů jsou **support vector machines**. Tak jim říkáme, když středové body jsou z trénovací množiny, tj:

$$f(x) = \sum_{i=1}^n k(x, x_i).$$

3. Metoda podpůrných vektorů (SVM)

Geometrický popis predikce pomocí lineární diskriminační funkce

- Mějme model s **lineární diskriminační funkcí**:

$$f(x) = \mathbf{w}^T \boldsymbol{\varphi}(x) + w_0,$$

kde pokud $f(x) \geq 0$, predikujeme $Y(x) = 1$ a pokud $f(x) < 0$, predikujeme $Y(x) = -1$.

- Rozhodovací hranice** je nadrovina v $\mathbb{R}^M$, což je řešení této rovnice:

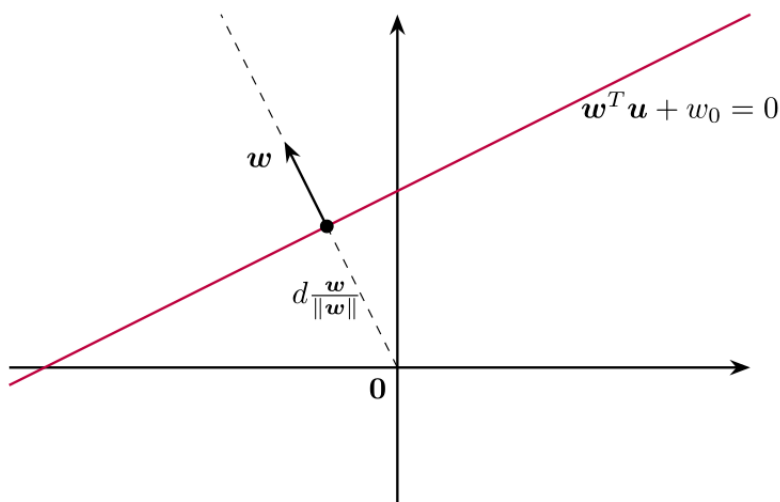
$$\mathbf{w}^T \mathbf{x} + w_0 = 0$$

- Její normála je dána vektorem $\mathbf{w}$.
- Její vzdálenost od počátku (ve směru) $\mathbf{w}$ je určena jako:

$$\mathbf{w}^T \mathbf{w} / \|\mathbf{w}\| + w_0 = \|\mathbf{w}\| + w_0 = 0,$$

což dává

$$= -w_0 / \|\mathbf{w}\|.$$



Znázornění rozhodovací hranice.

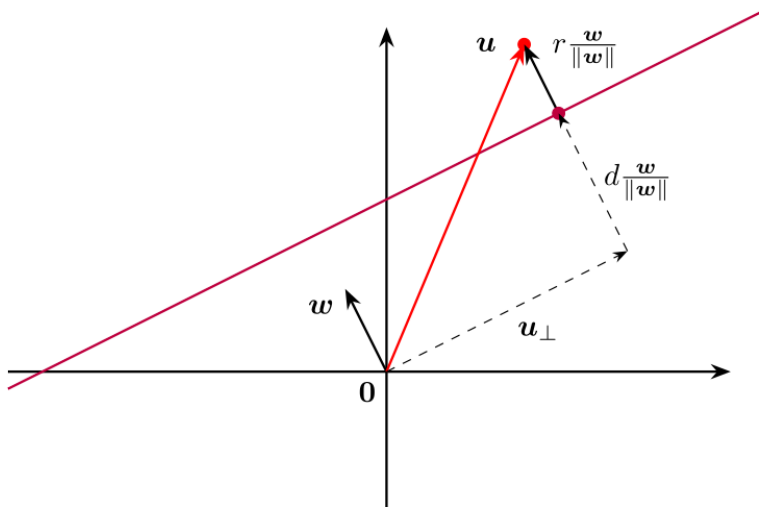
- Libovolný bod $\mathbf{x} \in \mathbb{R}^M$ lze **jednoznačně rozložit** na část, která je **kolmá** na $\mathbf{w}$ a část $\parallel$, která je **rovnoběžná** s $\mathbf{w}$,

$$\mathbf{x} = \mathbf{x}_{\perp} + \mathbf{x}_{\parallel}.$$

- Dále zavedeme $\mathbf{x}_{\perp} = \mathbf{x} - \mathbf{x}_{\parallel}$ a $\mathbf{x}_{\parallel} = \frac{\mathbf{w}^T \mathbf{x}}{\|\mathbf{w}\|^2} \mathbf{w}$, dostaneme:

$$\mathbf{x} = \mathbf{x}_{\perp} + \frac{\mathbf{w}^T \mathbf{x}}{\|\mathbf{w}\|^2} \mathbf{w}$$

Modrá část $+w\|w\|$ leží na rozhodovací hranici a červená část $w\|w\|$ odpovídá vzdálenosti od rozhodovací hranice ve směru w .



Vizualizace rozkladu $u = u_{\perp} + d \frac{w}{\|w\|} + r \frac{w}{\|w\|}$.

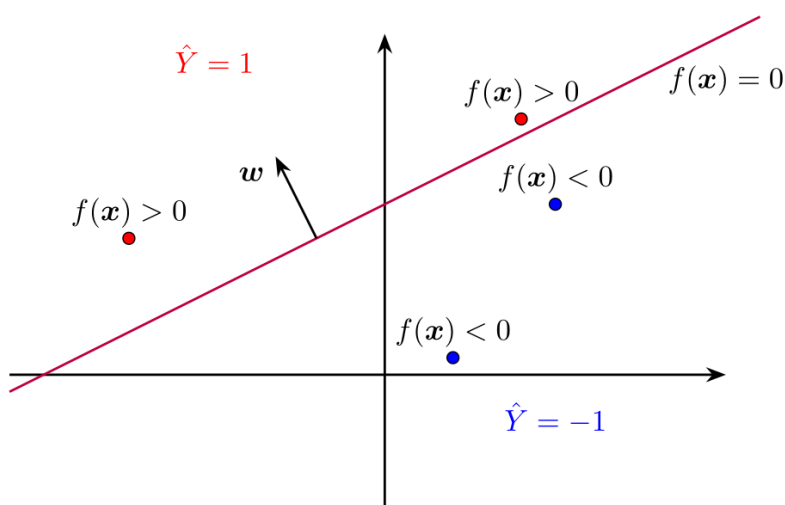
- Vzdálenost od rozhodovací hranice je

$$= w^T + w_0 \|w\|$$

a tedy pro $\varphi(x)$ je vzdálenost od rozhodovací hranice

$$= w^T \varphi(x) + w_0 \|w\| = (x) \|w\|.$$

- Navíc znaménko (x) rozděluje prostor $\mathbb{R}^M$ na dva pod-prostory a určuje predikci ve všech bodech následovně:
 - **Pozitivní** ve směru w : $(x) > 0 \quad Y(x) = 1$
 - **Negativní** ve směru $-w$: $(x) < 0 \quad Y(x) = -1$

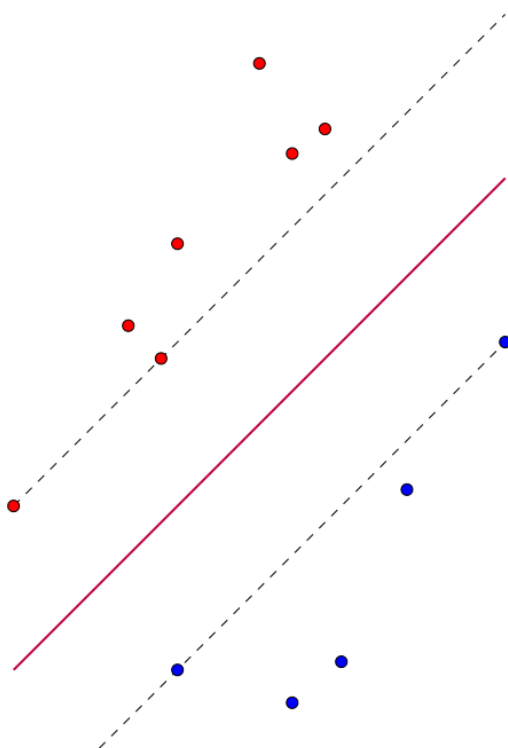


Rozhodovací hranice a predikční polo-prostory.

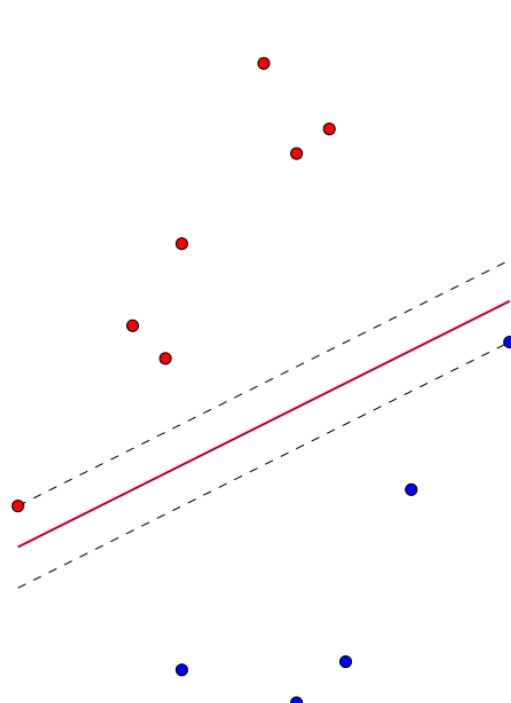
Metoda maximálního odstupu v lineárně separabilním případě

- Typicky existuje nekonečně mnoho řešení, které separuje dvě třídy. My vybereme takové, pro které je **vzdálenost separující nadroviny od nejbližšího bodu největší**

(maximální odstup).



Separující nadrovina s větším odstupem.



Separující nadrovina s menším odstupem.

- Na obrázku vidíme, že nadrovina **vlevo** je **výhodnější pro predikci**, protože nová data, blíže separující nadrovině, mají **větší pravděpodobnost být správně zařazena**.
- Chceme pro **bod s nejmenší vzdáleností** od separující nadroviny tuto **vzdálenost největší**.
- Tedy pro vzdálenost i -tého bodu od separující nadroviny $d_i = (x_i \cdot w) / \|w\|$ chceme najít w a w_0 , aby minimální d_i bylo největší možné, tedy:

$$\max_{w, w_0} \min_i Y_i (w^T \varphi(x_i) + w_0) / \|w\|$$

a navíc musí platit, aby byl každý bod na správné straně rozhodovací hranice: $Y_i (w^T \varphi(x_i) + w_0) \geq 0$.

- Při přeškálování parametrů $w \rightarrow \alpha w$ a $w_0 \rightarrow \alpha w_0$ se vzdálenost od hranice nezmění.
- Zvolíme tedy škálu takovou, aby

$$Y_i (w^T \varphi(x_i) + w_0) = 1$$

pro bod **nejblíže** rozhodovací hranici.

- Pro vyhovující w to znamená, že:

$$\min_i Y_i (w^T \varphi(x_i) + w_0) / \|w\| = 1 / \|w\|$$

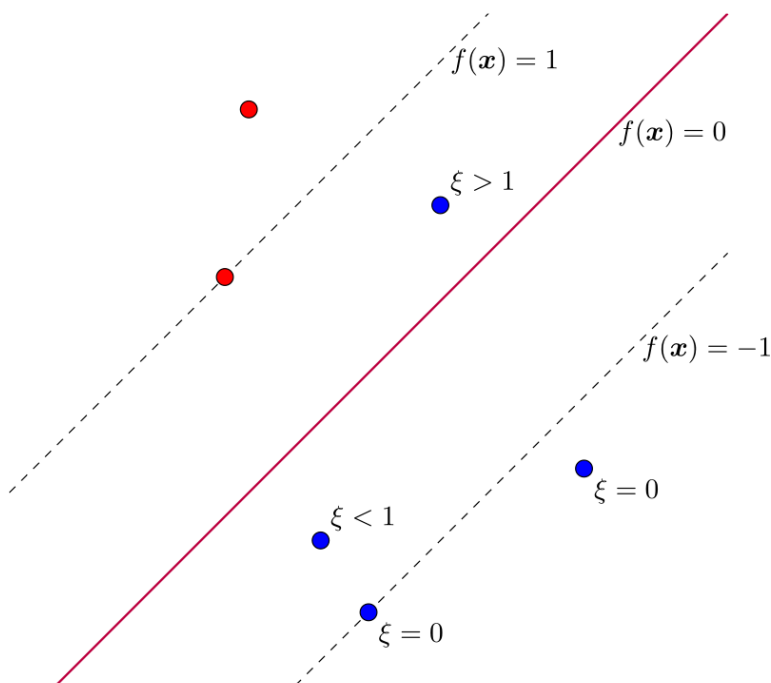
- Maximalizujeme tedy $\|w\|^{-1}$ což je ekvivalentní jako minimalizování $\|w\|^2$.
- Finální formulace je potom:

$$\min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad Y_i(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0) \geq 1 \quad \forall i.$$

- *Toto vede na problém **kvadratického programování**.*
- Body řešení, pro které platí $Y_i(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0) = 1$ se nazývají **podpůrné vektory**
- Metoda se tedy nazývá **metoda podpůrných vektorů** (angl. **support vector machine**).

Obecný lineárně neseparabilní případ

- Když data **nejsou lineárně separabilní**, nejsme schopni udržet vazbu $Y_i(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0) \geq 1$ pro všechny i .
- Rozvolníme vazby, aby aspoň některé body mohly tuto hranici překročit.
- Zavedeme **uvolněné proměnné (slack variables)** ξ_i následovně:
 - $\xi_i = 0$ – pokud bod i je na nebo vně pásu odstupů na správné straně.
 - $0 < \xi_i < 1$ – pokud bod i je uvnitř pásu odstupů, ale na správné straně.
 - $\xi_i \geq 1$ – pokud bod i je na špatné straně rozhodovací hranice.



Vztah uvolněných proměnných k pásu odstupů.

- Pomocí uvolněných proměnných můžeme rozvolnit podmínky pomocí **měkkých podmínek**:

$$Y_i(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0) \geq 1 - \xi_i \quad \forall i = 1, \dots, N.$$

- Přidáme ještě penalizační člen a dostáváme novou **optimalizační úlohu**:

$$\min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^N \xi_i$$

vzhledem k

$$\xi_i \geq 0 \quad Y_i(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0) \geq 1 - \xi_i \quad \forall i.$$

- Něco něco **Lagrangeovy multiplikátory a princip duality**...

- Řešení se nazývá **metoda podpůrných vektorů** (angl. **support vector machine**) (SVM)

Tvar výsledného jádrového modelu

- Řešením je **jádrový model**:

$$f(\mathbf{x}) = \sum_{i=1}^N k(\mathbf{x}, \mathbf{x}_i) + w_0$$

$$s = aY.$$

- Čím **větší** parametr a , tím **méně** podpůrných vektorů dostaneme.

4. Redukce dimenzionality pomocí ortogonální projekce

Problematika vysokých dimenzí

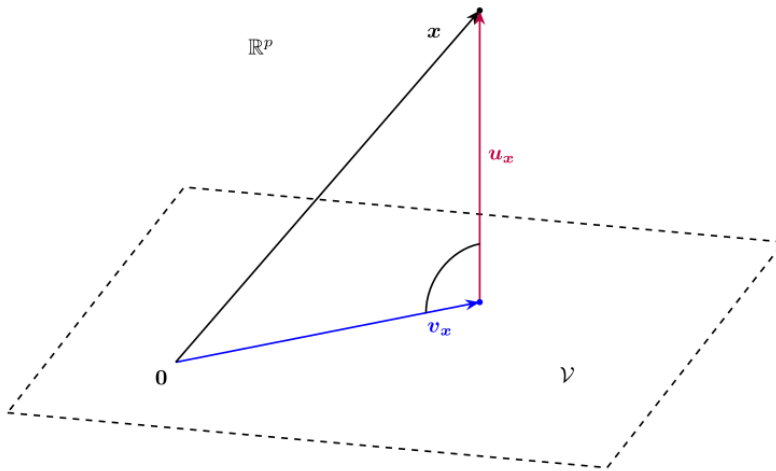
- **Prokletí dimenzionality**:
 - V jednotkové krychli v d rozměrném prostoru mějme podkrychli s hranou délky $(1 - \varepsilon)$, v ní máme $(1 - \varepsilon)^d$ 100 všech bodů. (*Předpokládáme rovnoměrné rozdělení.*)
 - pro $\varepsilon = 0.1$:
 - $d = 1$: $(1 - 0.1)^1 \cdot 100 = 90$ všech dat.
 - $d = 2$: $(1 - 0.1)^2 \cdot 100 = 81$ všech dat.
 - $d = 10$: $(1 - 0.1)^{10} \cdot 100 \approx 3.5$ všech dat.
 - $d = 100$: $(1 - 0.1)^{100} \cdot 100 \approx 0.002$ všech dat.
 - S rostoucí dimenzí **exponenciálně klesá hustota** bodů, tím pádem jsou body od sebe **hodně vzdáleny**.
 - Predikce jsou pak **méně spolehlivé** a model musí dělat **velké extrapolace**.
- Větší množství dat by to *teoreticky* vyřešilo, ale zase potřebujeme exponenciální zvětšování počtu dat s rostoucí dimenzí.
- Ve většině reálných případů se ale data nevyskytují rovnoměrně v prostoru, ale **vyskytují se v omezených oblastech prostoru menší dimenze**.
- Tomuto předpokladu říkáme **hypotéza variet** (angl. **manifold hypothesis**).
- Z této hypotézy dává smysl se pokoušet o **zobrazení dat do prostoru menší dimenze**.
- Rozdělíme metody na dva typy:
 - **Lineární projekce**
 - **Nelineární projekce**

Ortogonální projekce na q rozměrný podprostor

- **Lineární projekce**
- Budeme se snažit **projektovat data** z původního **příznakového prostoru** $\mathbb{R}^p$ do **dimenzionálního podprostoru** $\mathbb{R}^q$.
- Víme, že každý bod $\mathbf{x} \in \mathbb{R}^p$ se dá rozložit jako:

$$\mathbf{x} = \mathbf{x} + \mathbf{x},$$

kde $\mathbf{x}$ je bod vektorového prostoru a $\mathbf{x}$ je vektor kolmý na .



Bod $\mathbf{x}$ se nazývá **ortogonální projekce** bodu $\mathbf{x}$ na podprostor .

- **Ortogonální projekce pomocí ortonormální báze.**

- Uvažujme **ortonormální bázi** prostoru $\mathbb{R}^p$ $(\mathbf{v}_1, \dots, \mathbf{v}_p)$.
- Každý bod $\mathbf{x} \in \mathbb{R}^p$ lze napsat jako:

$$\mathbf{x} = x_1 \mathbf{v}_1 + \dots + x_p \mathbf{v}_p$$

kde i -tý koeficient $x_i = \mathbf{x}^T \mathbf{v}_i$.

- Pro ortogonální rozklad $\mathbf{x} = \mathbf{x} + \mathbf{x}$ tedy dostáváme:

$$\mathbf{x} = x_1 \mathbf{v}_1 + \dots + x_p \mathbf{v}_p \quad \mathbf{x} = x_1 \mathbf{v}_1 + \dots + x_p \mathbf{v}_p$$

- Projekci na podprostor reprezentujeme jako:

$$\mathbf{x} = \mathbf{V}^T \mathbf{x},$$

kde $\mathbf{V} \in \mathbb{R}^p$, která ve sloupcích napsané vektory $(\mathbf{v}_1, \dots, \mathbf{v}_p)$ ortonormální báze . Tedy:

$$\mathbf{V} = \begin{pmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \dots & \mathbf{v}_p \end{pmatrix}$$

Projekce jednoho bodu i celého datasetu

- **Projekce jednoho bodu:**

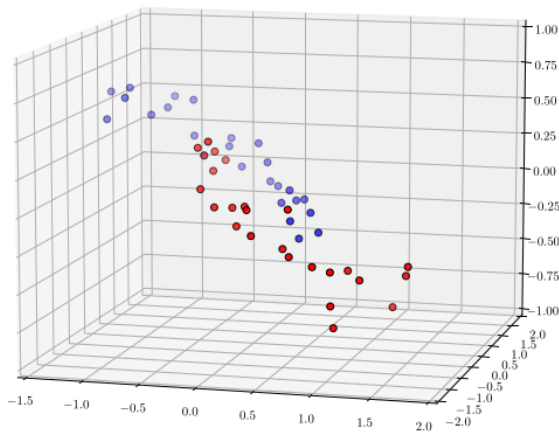
- Projekci jednoho bodu $\mathbf{x} \in \mathbb{R}^p$ realizujeme jako:

$$\mathbf{x} = \mathbf{V} \mathbf{V}^T \mathbf{x} \quad \text{no} \quad \mathbf{V}^T \mathbf{x} = \mathbf{x}^T \mathbf{V}$$

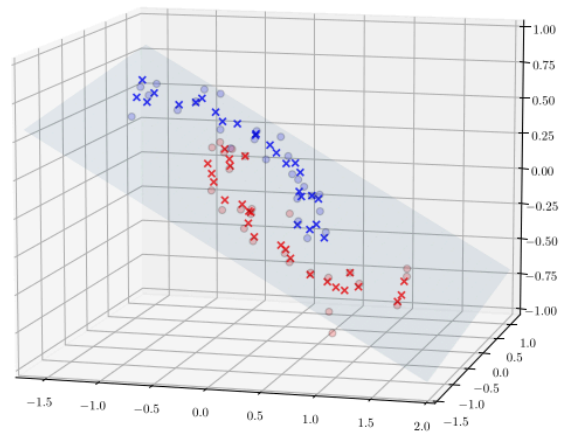
- **Projekce celého datasetu:**

- Projekci celého datasetu $\mathbf{X}$ realizujeme jako:

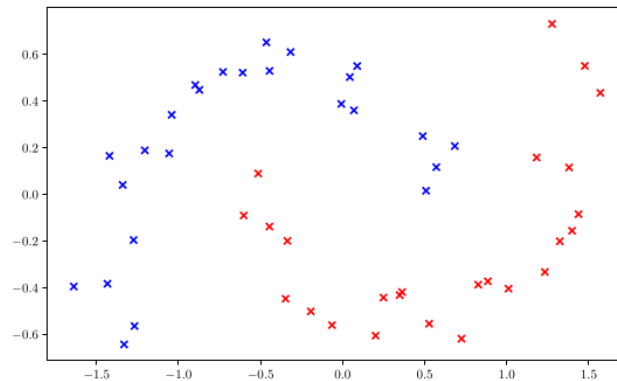
$$\mathbf{X} = \mathbf{X} \mathbf{V}^T, \quad \mathbf{X} \in \mathbb{R}^{N \times p},$$



Originální dataset $\mathbf{X}$



Projekce $\mathbf{X}_{\mathcal{V}}$ datasetu na podprostor $\mathcal{V}$



Transformovaný dataset $\mathbf{T}_q$

- **Pozorování:**

$$\|\mathbf{x}\|^2 = \|\mathbf{T}\mathbf{x}\|^2 = \mathbf{x}^{TTT}\mathbf{x}^T = \mathbf{x}^{TT}\mathbf{x} = \mathbf{x}^T\mathbf{x} = \|\mathbf{x}\|^2$$

Rovnost norem znamená, že **transformace do menšího prostoru nemění vzdálenosti od počátku**. Tedy nedochází k deformaci dat.

5. Analýza hlavních komponent (PCA)

- Jak najít pro **každé** příslušný podprostor ?

Optimalizační kritérium v metodě hlavních komponent

- Chceme, aby pro každé byla **minimalizovaná kvadratická chyba projekce**.
- To jde ruku v ruce s tím, že najdeme takový podprostor, pro který budou mít transformovaná data **největší rozptyl**.

Postup provedení metody hlavních komponent

- **Vystředujeme dataset:**

$$\mathbf{x}_i = \mathbf{x}_i - \mathbf{x}, \quad \mathbf{x} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i$$

$$\mathbf{X} = \begin{pmatrix} - & \mathbf{x}_1 & - \\ - & \mathbf{x}_2 & - \\ & \vdots & \\ - & \mathbf{x}_N & - \end{pmatrix}$$

- **Spočítáme varianční matici:**

$$= \frac{1}{N-1} \mathbf{X}^T \mathbf{X}, \quad \in \mathbb{R}^{p,p}$$

- **Uděláme spektrální rozklad této matice:**

- Dostaneme vlastní čísla, která ještě **seřadíme sestupně** (od největšího po nejmenší).
- Máme tedy **vlastní čísla** $(\lambda_1 \ \lambda_2 \ \dots \ \lambda_p)$ (pokud má vlastní číslo λ_i násobnost 1, uvažujeme ho *krát*)
- K nim dostaneme příslušné **vlastní vektory** $(\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_p)$ s normou $\|\mathbf{v}_i\| = 1$ pro všechna $i = 1, \dots, p$.

- **Vytvoříme podprostor :**

- Pro podprostor dimenze k vybereme **prvních vlastních vektorů** $(\mathbf{v}_1, \dots, \mathbf{v}_k)$ jako bázi tohoto podprostoru.
- Příslušná matice $\mathbf{V}$, která bude sloužit k transformaci do menší dimenze, pak vypadá následovně:

$$= \begin{pmatrix} & & \\ & 1 & 2 & \dots \\ & & & \end{pmatrix}$$

- **Vytvoření transformovaného datasetu:**

- Transformovaný dataset vytvoříme jako:

$$= \mathbf{X} \mathbf{V}$$

Hlavní komponenty a jejich statistická interpretace

- Uvažujme T_i jako i -tý sloupec matice $\mathbf{T}$, tedy:

$$T_i = \frac{1}{N-1} \mathbf{X}^T \mathbf{x}_i$$

- Tyto nové příznaky (sloupce T_i) nazýváme **hlavní komponenty**.
- **Statistická interpretace:**

- Co se stane s **průměrem**?

$$T_i = \frac{1}{N-1} \sum_{k=1}^N \mathbf{x}_k^T \mathbf{v}_i = \frac{1}{N-1} \sum_{k=1}^N \mathbf{x}_k^T \mathbf{v}_i = 0$$

zůstane stále 0.

- Jak je na tom nová **varianční matice**?

$$\mathbf{T} = \mathbf{1} \mathbf{1}^T - \mathbf{1} \mathbf{1}^T$$

tedy

$$\begin{aligned}
o(T_i, T) &= 1N - 1(T)_i, \\
&= 1N - 1({}^T \mathbf{X}^T \mathbf{X})_i, \\
&= 1N - 1_i^T \mathbf{X}^T \mathbf{X}_T \\
&= \lambda_i^T = \begin{matrix} 0 & o_i \\ \lambda & o_i \end{matrix} = .
\end{aligned}$$

- Platí tedy:
 - Rozptyl i -té hlavní komponenty je roven i -tému vlastnímu číslu λ_i .
 - Výběrem hlavních komponent tedy vybereme směry, ve kterých mají data **největší rozptyl**.
 - Různé komponenty jsou **nekorelované**.
- **Závěrečné poznámky:**
 - Zvolením p nezahodíme žádnou informaci, ale pouze přejdeme do **reprezentace dat v ortonormální bázi**.
 - Pro spektrální rozklad se používá SVD rozklad, který je numericky stabilnější.
 - PCA se soustředí na **globální vlastnosti** namísto **lokálních vztahů**.

6. Manifold learning a lokálně lineární vnoření

Problematika vysokých dimenzí

viz. Otázka 4.

Hypotéza variet

viz Otázka 4.

Lokálně lineární vnoření

- Sleduje, jak jednotlivé body z trénovací množiny **závisí lineárně na svém nejbližším okolí**.
- Následně **hledá méně dimenzionální reprezentaci**, která tyto **vztahy zachovává**.
- **Postup:**
 - Uvažujeme i -tý body trénovací množiny $\mathbf{x}_i$.
 - Pro dané k nalezneme jeho k nejbližších sousedů.
 - Dále najdeme váhy w_i , takové, že:
 - $w_i = 0$ pro takové, že $\mathbf{x}$ není mezi k nejbližšími sousedy $\mathbf{x}_i$.
 - $w_i = 1$,
 - vzdálenost mezi $\mathbf{x}_i$ a $w_i \mathbf{x}$ byla nejmenší možná.
 - Toto uděláme pro všechny body trénovací množiny. Ekvivalentně hledáme:

2

$$\min_{w_i} \sum_{i=1}^N \|\mathbf{x}_i - w_i \mathbf{x}\|^2$$

- Označme matici takovou, že obsahuje optimální hodnoty vah w_i .

- V dalším kroku hledáme pro každé $\mathbf{x}_i \in \mathbb{R}^p$ rozměrnou reprezentaci $i \in \mathbb{R}$ takovou, že je vzdálenost mezi i a w_i , nejmenší možná.
- Takto získáme matici, která má v řádcích zapsané výsledné optimální hodnoty i .
- **Shrnutí:**
 - **Nalezení vah:**

$$w_i = \arg \min_{w_i} \sum_{i=1}^N \|\mathbf{x}_i - w_i\|^2$$

Lidsky: Hledáme množinu všech optimálních vah w_i .

- **Nalezení vnoření:**

$$i = \arg \min_i \sum_{i=1}^N \|\mathbf{x}_i - w_i\|^2$$

Lidsky: Hledáme množinu všech nových vektorů i , přičemž používáme už nalezené optimální váhy w_i .

7. Naivní Bayesův klasifikátor

Klasifikace pomocí podmíněné pravděpodobnosti

- Chceme predikovat ($Y = \mathbf{X} = \mathbf{x}$).
- Místo přímé konstrukce ale použijeme **Bayesovu větu**:

$$P(Y = \mathbf{x} | \mathbf{X} = \mathbf{x}) = P(\mathbf{X} = \mathbf{x} | Y = \mathbf{x}) P(Y = \mathbf{x}) / P(\mathbf{X} = \mathbf{x})$$

- Tedy *nějak* odhadneme ($\mathbf{X} = \mathbf{x} | Y = \mathbf{x}$).
- Jak na to:
 - Zajímá nás argument maxima vzhledem k, tedy zahodíme ($\mathbf{X} = \mathbf{x}$). (Je pro všechny hodnoty stejný.)
 - Pro predikci tedy dostáváme:

$$Y = \arg \max_{Y} P(\mathbf{X} = \mathbf{x} | Y = \mathbf{x}) P(Y = \mathbf{x})$$

- Tomuto odhadu se říká **MAP odhad** (z angl. **maximum a posteriori estimate**).
- Ted' už jen *nějak* odhadneme ($\mathbf{X} = \mathbf{x} | Y = \mathbf{x}$).

Naivní Bayesův klasifikátor

- **Předpoklad:**
 - **Za podmínky** $Y = \mathbf{x}$ jsou všechny příznaky nezávislé.
- Z tohoto předpokladu plyne, že:

$$P(\mathbf{X} = \mathbf{x} | Y = \mathbf{x}) = \prod_{i=1}^p P(X_i = x_i | Y = \mathbf{x})$$

- **Naivita** plyne z toho, že předpokládáme, že pro **fixní hodnotu** Y jsou příznaky nezávislé.
- Výsledný **MAP odhad** naivního Bayesova klasifikátoru je tedy:

$$Y = \underset{g \in \mathcal{Y}}{\arg \max} \prod_{i=1}^n (X_i = x_i | Y = g)$$

- *Poznámka: Ačkoliv je v praxi zmíněný předpoklad velmi hrubý a naivní, ukazuje se, že model v mnoha případech dává až překvapivě dobré výsledky.*

Modely podmíněných pravděpodobností

- **Binární veličina X :**
 - V tomto případě je vhodný model pro rozdělení $XY =$ **Bernoulliho rozdělení** s parametrem p .
 - Označujeme $(XY =) (p)$.
 - Pro pravděpodobnosti platí:
 $(X = 1 | Y =) = p \quad (X = 0 | Y =) = 1 - p$
 - Odhad parametru p :

$$\hat{p} = \frac{N_{11} + N_{01}}{N_{11} + N_{01} + N_{10} + N_{00}}$$

kde $N_{i,}$ značí počet dat pro $X = i$ a $Y = .$

- Jedná se o **maximálně věrohodnostní odhad** (MLE).
- Je však jasné, že pokud dojde k $N_{0,} = 0$ nebo $N_{1,} = 0$, celá pravděpodobnost zdegeneruje na $p = 0$ (resp. 1).
- To ale znamená násobení nulou v MAP odhadu a celý odhad spadne na nulu. *Zero-frequency problem.*
- To se dá vyřešit **Bayesovským přístupem**:

Ve zkratce:

- Zavedeme **apriorní rozdělení** - jak si myslíme, že bude parameter p vypadat. (U mince, třeba 12).
- Na základě napozorovaných dat pak určíme **aposteriorní rozdělení**:

$$p_p(\mathbf{x}) = (\mathbf{X} = \mathbf{x} | p) p(p)$$

kde $(\mathbf{X} = \mathbf{x})$ je pravděpodobnost, že napozorujeme $\mathbf{X} = \mathbf{x}$ pokud p je správný parametr a

$$p_p(\mathbf{x}) = \prod_{i=1}^n (X_i = x_i | p) p$$

- Bodový odhad p se získá spočtením střední hodnoty:

$$\hat{p} = \int p \, p_p(\mathbf{x}) \, d\mathbf{x}$$

- V případě Bernoulliho rozdělení se bere jako apriorní rozdělení **Beta rozdělení**.

- Výsledný odhad z apriorního rozdělení je pak:

$$p = N_1 + 1N_1 + N_0 + 2$$

- Tomuto se říká **add-one smoothing (Laplace smoothing)**.

- **Kategorická veličina X :**

- Nechť X nabývá k různých hodnot $1, \dots, k$.
- Vhodným modelem je potom **kategorické rozdělení**.
- Zde máme $\mathbf{p} = (p_1, \dots, p_k)^T$, kde $p_1, \dots, p_k$ jsou podmíněné pravděpodobnosti hodnot $1, \dots, k$.
- **Odhad k -rozměrného parametru** se využívá:

$$\mathbf{p} = (p_1, \dots, p_k)^T \quad p_i = N_i + 1N_1 + \dots + N_k + k$$

- **Spojité veličina X :**

- Místo podmíněné pravděpodobnosti se vezme **podmíněná hustota** $p_X(x)$.
- Této hustotě odpovídá **distribuční funkce**:

$$F_X(x) = P(X \leq x)$$

- Predikci provedeme pomocí MAP odhadu:

$$Y = \underset{i=1}{\overset{l}{\arg \max}} (X_i = x_i | Y =) \quad p_{i=l+1} X_i(x_i) | (Y =),$$

kde $X_1, \dots, X_l$ jsou diskrétní příznaky a $X_{l+1}, \dots, X_p$ jsou spojité příznaky.

- **Gaussovo rozdělení:**

- Předpokládejme, že $(X, Y) \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$.
- Použijeme MLE odhady:

$$\hat{\boldsymbol{\mu}} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \quad \hat{\Sigma} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \hat{\boldsymbol{\mu}})(\mathbf{x}_i - \hat{\boldsymbol{\mu}})^T,$$

kde $\mathbf{x}_i$ jsou hodnoty příznaku $\mathbf{X}$, pro které $Y = y$.

8. Lineární diskriminační analýza (LDA)

Generativní versus diskriminativní modely

- **Generativní model:**

- Odhadujeme podmíněnou pravděpodobnost $p(\mathbf{X} = \mathbf{x} | Y = y)$.
- Společně pak s $p(Y = y)$ dostáváme sdruženou pravděpodobnost:

$$p(\mathbf{X} = \mathbf{x}, Y = y) = p(\mathbf{X} = \mathbf{x} | Y = y)p(Y = y).$$

- Tento přístup, kdy vytváříme sdruženou pravděpodobnost $p(\mathbf{X} = \mathbf{x}, Y = y)$ se nazývá **generativní přístup** a výsledný model je pak **generativní model**.
- Známe úplnou informaci o rozdělení, ze kterého byla data generována a jsme schopni generovat sami nová data.

- **Diskriminativní model:**

- Druhou možností je odhadovat z trénovacího datasetu podmíněnou pravděpodobnost ($Y = \mathbf{X} = \mathbf{x}$).
- Tomuto přístupu se říká **diskriminativní**.

Log-sum-exp trik

- Počítat

$$(Y = \mathbf{X} = \mathbf{x}) = (\mathbf{X} = \mathbf{x} | Y =)_{\in} (\mathbf{X} = \mathbf{x} | Y =) (Y =)$$

může vést k **numerickému podtečení**.

- Proto spočítáme logaritmus celé pravděpodobnosti

$$\ln(Y = \mathbf{X} = \mathbf{x}) = \ln(\mathbf{X} = \mathbf{x} | Y =) + \ln(Y =) \\ - \ln_{\in} (\mathbf{X} = \mathbf{x} | Y =) (Y =)$$

- Uděláme následující úpravu

$$\ln_{\in} (\mathbf{X} = \mathbf{x} | Y =) (Y =) = \ln_{\in}^{-b},$$

kde $b = -\ln(\mathbf{X} = \mathbf{x} | Y =) - \ln(Y =)$

- **Log-sum-exp trik:**

$$\ln(-^{120} + -^{121}) = \ln(-^{120}(^0 + ^{-1})) = \ln(^0 + ^{-1}) - 120$$

což už je numericky stabilní.

- V našem případě dostaneme:

$$\ln_{\in}^{-b} = \\ \ln_{\in}^{-b} = \\ - + \ln_{\in}^{-b}$$

kde $= \ln_{\in} b$.

(Gaussovská) diskriminační analýza

- Modelujeme pomocí **vícerozměrného normálního rozdělení**.

- **Definice:**

Bud' $\in \mathbb{R}^p$ vektor středních hodnot a $\in \mathbb{R}^{p,p}$ symetrická a **pozitivně semidefinitní** matice. Náhodný vektor $\mathbf{X} = (X_1, \dots, X_p)^T$ má **p -rozměrné normální rozdělení** s parametry a , píšeme $\mathbf{X} (,)$, jestliže pro každé $\in \mathbb{R}^p$ platí $^T \mathbf{X} (^T, ^T)$.

- **Věta:**

Bud' $\in \mathbb{R}^p$ vektor a $\in \mathbb{R}^{p,p}$ symetrická a **pozitivně definitní** matice. Potom $\mathbf{X} (,)$ právě tehdy, když $\mathbf{X}$ má *spojité sdružené rozdělení s hustotou*:

$$\mathbf{x}(\mathbf{x}) = 1(2)^{p212-12(\mathbf{x}-)^{T-1}(\mathbf{x}-)}$$

- Podmíněné sdružené rozdělení tedy modelujeme vícerozměrným normálním rozdělením:

$$\mathbf{X}Y = (\mathbf{x}, y)$$

- MAP odhad je dán vztahem:

$$Y = \underset{\mathbf{x}}{\arg \max} \mathbf{P}(Y=y|\mathbf{X}=\mathbf{x}).$$

Lineární diskriminační analýza

- Kvadratická diskriminační analýza:**

- MAP odhad Y maximalizuje pravděpodobnosti

$$\mathbf{P}(Y=y|\mathbf{X}=\mathbf{x}) = \mathbf{P}(\mathbf{X}=\mathbf{x}|Y=y) \mathbf{P}(Y=y) \underset{\mathbf{x}}{\arg \max} \mathbf{P}(\mathbf{X}=\mathbf{x}|Y=y).$$

- Dosadíme-li předpis pro sdruženou hustotu, dostaneme:

$$\mathbf{P}(Y=y|\mathbf{X}=\mathbf{x}) = (2\pi)^{-p/2} |\Sigma_y|^{-1/2} \exp\left(-\frac{1}{2}(\mathbf{x}-\mu_y)^T \Sigma_y^{-1} (\mathbf{x}-\mu_y)\right) \underset{\mathbf{x}}{\arg \max} (2\pi)^{-p/2} |\Sigma_y|^{-1/2} \exp\left(-\frac{1}{2}(\mathbf{x}-\mu_y)^T \Sigma_y^{-1} (\mathbf{x}-\mu_y)\right),$$

kde $\mu_y = \mathbf{E}(Y=y)$.

- Při MAP odhadu pro y s největší pravděpodobností se porovnávají velikosti čítelů a v důsledku z pohledu $\mathbf{x}$ tedy hodnoty **kvadratické** členy $(\mathbf{x}-\mu_y)^T \Sigma_y^{-1} (\mathbf{x}-\mu_y)$.

- Lineární diskriminační analýza:**

- Je speciální případ **kvadratické diskriminační analýzy**, kde předpokládáme, že **varianční matice jsou pro všechny třídy stejné**: $\Sigma_y = \Sigma$.
- Tedy pro MAP odhad máme v čitateli:

$$\begin{aligned} (2\pi)^{-p/2} |\Sigma|^{-1/2} \exp\left(-\frac{1}{2}(\mathbf{x}-\mu_y)^T \Sigma^{-1} (\mathbf{x}-\mu_y)\right) &= \\ (2\pi)^{-p/2} |\Sigma|^{-1/2} \exp\left(-\frac{1}{2}(\mathbf{x}-\mu_y)^T \Sigma^{-1} (\mathbf{x}-\mu_y)\right) &= \\ (2\pi)^{-p/2} |\Sigma|^{-1/2} \exp\left(-\frac{1}{2}(\mathbf{x}-\mu_y)^T \Sigma^{-1} (\mathbf{x}-\mu_y)\right) &= \end{aligned}$$

- Člen $(2\pi)^{-p/2} |\Sigma|^{-1/2}$ nezávisí na y a ve zlomku se zkrátí.
- Zavedeme:

$$W_y = -\frac{1}{2}(\mathbf{x}-\mu_y)^T \Sigma^{-1} (\mathbf{x}-\mu_y) = -\frac{1}{2} \mathbf{x}^T \Sigma^{-1} \mathbf{x} + \mathbf{x}^T \Sigma^{-1} \mu_y - \frac{1}{2} \mu_y^T \Sigma^{-1} \mu_y$$

a můžeme psát:

$$\mathbf{P}(Y=y|\mathbf{X}=\mathbf{x}) = \exp(W_y) \underset{\mathbf{x}}{\arg \max} \exp(W_y).$$

- z MAP pohledu se jedná o porovnání čítelů a v důsledku toho porovnání výrazů **lineárních** v $\mathbf{x}$.
- Pro rozhodovací hranici mezi třídami y a z platí: $\mathbf{P}(Y=y|\mathbf{X}=\mathbf{x}) = \mathbf{P}(Y=z|\mathbf{X}=\mathbf{x})$

$\begin{aligned} \end{aligned}$

$\mathbf{P}(Y=y|\mathbf{X}=\mathbf{x}) = \mathbf{P}(Y=z|\mathbf{X}=\mathbf{x})$

$= \mathbf{P}(Y=y|\mathbf{X}=\mathbf{x})$

$\mathbf{P}(Y=y|\mathbf{X}=\mathbf{x}) = \mathbf{P}(Y=z|\mathbf{X}=\mathbf{x})$

$$(\beta_y - \beta_{y'})^T \mathbf{x} = \gamma_{y'} - \gamma_y$$

$$\mathbf{a} \in \mathbb{R}^p.$$

- Při binární klasifikaci dostaneme:
$$P(Y=1 | \mathbf{X} = \mathbf{x}) = \frac{e^{\beta_1^T \mathbf{x} + \gamma_1}}{e^{\beta_1^T \mathbf{x} + \gamma_1} + e^{\beta_0^T \mathbf{x} + \gamma_0}} = \sigma(\mathbf{w}^T \mathbf{x} + w_0),$$

$$w = 1 - 0, w_0 = 1 - 0.$$

- Pro odhad parametrů opět využíváme MLE:

$$\mathbf{N} = N \mathbf{N}, \quad \mathbf{N} = 1 N_{i=1} \mathbf{x}_i.$$

$$= 1 N \left(\sum_{i=0} (\mathbf{x}_i - \mathbf{0})(\mathbf{x}_i - \mathbf{0})^T + \sum_{i=1} (\mathbf{x}_i - \mathbf{1})(\mathbf{x}_i - \mathbf{1})^T \right)$$

Fisherova lineární diskriminační analýza

- Budeme chápat $(\mathbf{w}^T \mathbf{x} + w_0)$ jako **ortogonální projekci** do směru $\mathbf{w}$ posunutou o w_0 .
- Cílem je udělat **projekci na podprostor menší dimenze** tak, aby transformovaná data byla co **nejpříznivější pro predikci**.
- Od LDA se liší tím, že není formulována generativním přístupem a tedy neobsahuje požadavky na rozdělení.
- **Binární klasifikace:**
 - Jednorozměrný podprostor můžeme určit vektorem $\mathbf{w}$ a ortogonální projekce bodu $\mathbf{x}_i$ na něj pak odpovídá transformaci

$$y_i = \frac{\mathbf{w}^T \mathbf{x}_i}{\|\mathbf{w}\|}.$$

- Dále zavedeme projekce výběrových průměrů (geometrických středů) z jednotlivých tříd jako $\bar{\mathbf{y}} = \frac{1}{N} \sum \mathbf{w}^T \mathbf{x}_i$.
- Rozptyl projektovaných bodů v rámci jedné třídy můžeme (až na konstantu) vyjádřit jako

$$s_i^2 = \frac{1}{N_i} \sum (\mathbf{x}_i - \bar{\mathbf{x}})^2.$$

- Cílem FLDA je najít vektor $\mathbf{w}$, který **maximalizuje vzdálenost** mezi průměry projektovaných tříd a zároveň drží **rozptyl** obou tříd **malé**.
- Toto můžeme vyjádřit pomocí účelové funkce

$$J(\mathbf{w}) = (1 - 0)^2 + \frac{2}{N}$$

- Když zavedeme **rozptylovou matici mezi třídami**

$$S = (1 - 0)(1 - 0)^T$$

- a rozptylovou matici v rámci třídy

$$= \sum_{i_i=0} (\mathbf{x}_i - \mathbf{0})(\mathbf{x}_i - \mathbf{0})^T + \sum_{i_i=1} (\mathbf{x}_i - \mathbf{1})(\mathbf{x}_i - \mathbf{1})^T,$$

- tak můžeme získat účelovou funkcí jako

$$J(\mathbf{w}) = \mathbf{w}^T \mathbf{w} \mathbf{w}^T \mathbf{w}$$

- Pomocí gradientu pak lze ukázat, že maximum nastává, když

$$\mathbf{w} = \lambda \mathbf{w}, \quad \lambda = J'(\mathbf{w})$$

- což dává rovnici pro vlastní vektor

$$-\mathbf{1} \mathbf{w} = \lambda \mathbf{w}$$

- Lze vidět, že

$$\mathbf{w} = (\mathbf{1} - \mathbf{0})(\mathbf{1} - \mathbf{0}) \|\mathbf{w}\|$$

- Dosazením do předchozího vztahu

$$\lambda \mathbf{w} = -\mathbf{1}(\mathbf{1} - \mathbf{0})(\mathbf{1} - \mathbf{0}) \|\mathbf{w}\|$$

- což až na konstantu je rovno

$$\mathbf{w} = N^{-1}(\mathbf{1} - \mathbf{0})$$

- To je **totožný vztah jako u LDA**, kde $\mathbf{w} = N^{-1}(\mathbf{1} - \mathbf{0})$, protože $N = N$

- **Klasifikace do více tříd:**

- V případě tříd hledáme matici .
- Účelová funkce je

$$J(\mathbf{w}) = (\mathbf{w}^T)(\mathbf{w})$$

- Rozptylové matice mezi třídami i rozptylové matice v rámci třídy jsou definovány následovně

$$\begin{aligned} &= \sum_{i \in C} (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T \\ &= \sum_{i \in C} (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T \end{aligned}$$

9. Perceptron

Definice perceptronu

- **Definice:**

Perceptron je nejjednodušší model dopředné neuronové sítě. Sestává pouze z jednoho neuronu.

Na vstupu mějme vektor $\mathbf{x} = (x_1, \dots, x_p)$, k němu příslušné váhy $\mathbf{w} = (w_1, \dots, w_p)$ a

intercept (**bias**) w_0 .

Výstup perceptronu se získá aplikací **skokové aktivační funkce** na **vnitřní potenciál**.

Skoková aktivační funkce:

$$\sigma(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Vnitřní potenciál:

$$z = w_0 + \sum_{i=1}^p w_i x_i = \mathbf{w}^T \mathbf{x} + w_0$$

Neuron je tedy aktivován, pokud $\mathbf{w}^T \mathbf{x} \geq -w_0$, $-w_0$ se někdy nazývá **prahová hodnota**.

Algoritmus trénování perceptronu a jeho konvergence

• Algoritmus trénování

- Postupně procházíme body trénovací množiny
- Pro daný bod uděláme predikci a provedeme:

$$\begin{aligned} o &= Y - \hat{Y}, \\ w_i &\leftarrow w_i + o x_i, \quad i = 1, \dots, p, \\ w_0 &\leftarrow w_0 + o, \end{aligned}$$

kde $\hat{Y} = (\mathbf{w}^T \mathbf{x} + w_0)$ je predikce v bodě $\mathbf{x}$ a Y je skutečná hodnota.

- Algoritmus ukončíme, jakmile nemáme chybu pro žádný bod.
- **Věta o konvergenci**

Předpokládejme lineárně separabilní data, tj.

$$(\mathbf{w} \in \mathbb{R}^{p+1}, \|\mathbf{w}\| = 1) \quad (0) \quad (i \in 1, \dots, N) : \\ (2Y_i - 1) \mathbf{x}_i^T \mathbf{w} \geq \gamma$$

Dále předpokládejme, že pro nějaké $\gamma > 0$ platí $\|\mathbf{x}_i\| \leq 1$ pro všechna $i = 1, \dots, N$.

Potom počet kroků algoritmu trénování perceptronu, při kterých nastane chyba, je nejvýše $\frac{1}{\gamma^2}$.

Poznámky a značení:

$(\mathbf{w} \in \mathbb{R}^{p+1}, \|\mathbf{w}\| = 1)$: $\mathbf{w} = (w_0, w_1, \dots, w_p)^T$, tedy hledáme (zda existuje) vektor vah + bias, jednotkové délky, pro který bude platit ta podmínka.

(0) : Hledáme $\gamma > 0$, to představuje šířku mezery mezi třídami. Proto $\gamma > 0$.

$(i \in 1, \dots, N)$: N je počet bodů trénovací množiny, tedy budeme kvantifikovat pro všechny body trénovací množiny.

$(2Y_i - 1) \mathbf{x}_i^T \mathbf{w}$: $(2Y_i - 1)$ pouze převádí hodnoty z $Y_i \in \{0, 1\} \rightarrow \{-1, 1\}$. Výraz $\mathbf{x}_i^T \mathbf{w}$ představuje hodnotu vnitřního potenciálu pro ideální váhy, a protože $\|\mathbf{w}\| = 1$, představuje tento součin **geometrickou vzdálenost** bodu $\mathbf{x}_i$ od rozhodovací hranice. A tato vzdálenost od rozhodovací hranice musí být větší (nějaké kladné číslo). Bod tedy nesmí ležet přímo na rozhodovací hranici.

$(0)(\|\mathbf{x}_i\|)$: Můžeme si představit jako poloměr nejmenší možné koule se středem

v , do které se vejdou všechny trénovací body.

²² : Výsledný zlomek nám udává, *jak dlouho (nejdéle) potrvá trénování*.

Čím **větší** (čím více jsou data *roztahána* od středu), tím **déle trénování potrvá**.

Naopak čím **větší** (čím dále jsou od sebe třídy), tím **kratší dobu trénování potrvá**.

Když zase bude **malé**, třídy jsou k sobě blízko a **trénování tedy potrvá déle**.

Problémy perceptronu

- Dokáže řešit pouze **lineárně separabilní** problémy.
- Jednoduchý příklad, který perceptron nedokáže vyřešit, je funkce **XOR**.
- K vyřešení XORu stačí přitom vzít tři perceptrony, tomuto přístupu pak odpovídá použití **vícevrstvé neuronové sítě**.

10. Vícevrstvé neuronové sítě - aktivační funkce, dopředný a zpětný chod

- Vícevrstvé neuronové sítě jsou základním rozšířením jednoho neuronu.

Matematický popis vícevrstvé neuronové sítě

- Označíme $1, \dots, l$ počty neuronů v jednotlivých vrstvách.
- Na i -té vrstvě mějme výstup i -tého neuronu: $(i) : \mathbb{R}^{i-1} \rightarrow \mathbb{R}$.
- (i) se opět počítá jako $(\mathbf{w}^T \mathbf{x} + w_0)$, kde $\cdot$ je aktivační funkce daného neuronu a $w_0, w_1, \dots, w_{l-1}$ jsou jeho váhy.
- i -tou vrstvou pak chápeme jako vícehodnotovou funkci $(i) : \mathbb{R}^{i-1} \rightarrow \mathbb{R}^i$, kde $(i) = \left(\begin{smallmatrix} (i) \\ 1 \end{smallmatrix}, \dots, \begin{smallmatrix} (i) \\ i \end{smallmatrix} \right)^T$.
- **Celá neuronová síť** je pak reprezentovaná funkcí $\cdot : \mathbb{R}^0 \rightarrow \mathbb{R}^l$

$$\cdot = \begin{pmatrix} (l) \\ (l-1) \\ \dots \\ (2) \\ (1) \end{pmatrix}.$$

zkráceně

$$\begin{matrix} l \\ i=1 \end{matrix} \begin{pmatrix} (i) \\ 1 \end{pmatrix} \text{ no } \begin{matrix} l \\ i=1 \end{matrix} \begin{pmatrix} (l-i+1) \\ 1 \end{pmatrix}$$

Aktivační funkce pro výstupní i skryté vrstvy

- **Skryté vrstvy**
 - V případě i -té skryté vrstvy se nejdřív spočítá vektor lineárních kombinací z vektoru vstupů z předchozí vrstvy

$$= \mathbf{x}^T + \mathbf{w}_0.$$

- Potom se na každou složku i aplikuje nelineární aktivační funkce, která je skoro všude diferencovatelná (a hlavně někde nemá nulový gradient).
- **Nejčastější příklady:**

- **ReLU** (rectified linear unit)

$$f(x) = \max(0, x)$$

- **Leaky ReLU**

$$f(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0.01x & \text{if } x < 0 \end{cases}$$

na rozdíl od ReLU má nenulový gradient v záporných hodnotách.

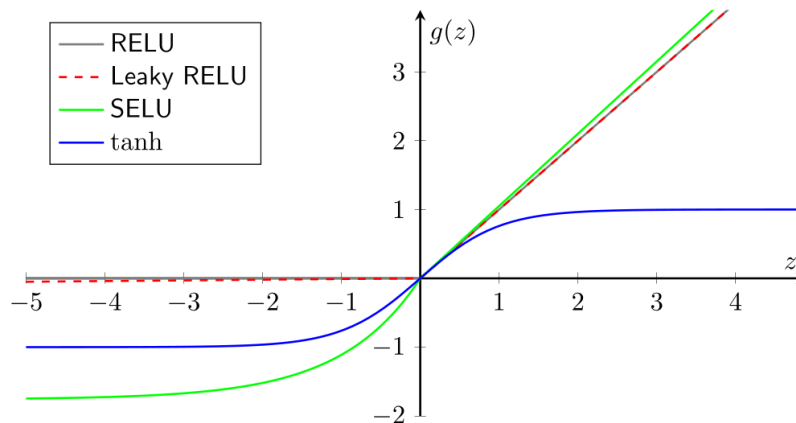
- **SELU** (scaled exponential linear unit)

$$f(x) = \lambda \begin{cases} x & \text{if } x \geq 0 \\ \exp(-1)x & \text{if } x < 0 \end{cases}$$

- **Hyperbolický tangens**

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Nejpoužívanější aktivační funkce



- Poslední skrytá vrstva produkuje vektor $\mathbf{z}_{l-1}$ skrytých příznaků.
- Na výstupní vrstvě se pak spočítá vektor lineárních kombinací

$$\mathbf{y} = \mathbf{z}_{l-1}^T \mathbf{w}_0$$

kde i -tý sloupec matice $\mathbf{w}_0 \in \mathbb{R}^{l-1, l}$ a i -tá složka $\mathbf{w}_0 \in \mathbb{R}^l$ jsou váhy i -tého neuronu.

- Poté se na $\mathbf{y}$ aplikuje výstupní aktivační funkce f .

- **Výstupní vrstva**

Podle typu úlohy aplikujeme vhodnou aktivační funkci

- **Regresní úloha**

- Ve výstupní vrstvě použijeme jeden neuron s **aktivační funkcí identity** $f(x) = x$.
- Výstupem je tedy

$$\mathbf{Y} = \mathbf{w}^T + w_0.$$

- **Binární klasifikace**

- Ve výstupní vrstvě se použije jeden neuron s **aktivační funkcí logistické sigmoidy**

$$\hat{y} = \hat{p} = 11 + \dots$$

- Výstup potom, stejně jako u logistické regrese, interpretujeme jako pravděpodobnost příslušnosti ke třídě 1

$$p(Y = 1 | \mathbf{X} = \mathbf{x}) = (\mathbf{w}^T \mathbf{x} + w_0).$$

- Predikce v bodě $\mathbf{x}$ je potom

$$Y = \begin{cases} 1 & \text{if } p(Y = 1 | \mathbf{X} = \mathbf{x}) \geq 0.5, \\ 0 & \text{in.} \end{cases}$$

- **Klasifikace do tříd**

- Ve výstupní vrstvě se používá neuronů s normovanou **aktivační funkcí softmax**

$$so_i(\mathbf{z}) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}.$$

- Výstupní hodnota $so_i(\mathbf{z})$ i -tého neuronu interpretujeme jako pravděpodobnost příslušnosti ke třídě i , tj.

$$p(Y = i | \mathbf{X} = \mathbf{x}) = so_i(\mathbf{w}^T \mathbf{x} + \mathbf{w}_0).$$

- K predikci je vybrána třída s největší pravděpodobností

$$Y = \underset{i \in \{1, \dots, n\}}{\arg \max} p(Y = i | \mathbf{X} = \mathbf{x}).$$

Používané ztrátové funkce

- **Regresní úloha**

Typickou volbou je **squared error**

$$L(Y, \hat{Y}) = (Y - \hat{Y})^2.$$

Další možností je L_1 ztrátová funkce **absolute error**

$$L(Y, \hat{Y}) = |Y - \hat{Y}|.$$

- **Binární úloha**

Používá se **binary cross-entropy loss**

$$L(Y, p) = -Y \log p - (1 - Y) \log(1 - p).$$

- **Kategorická úloha**

Používá se **categorical cross-entropy loss**

$$L(Y, p) = - \sum_{i=1}^n \mathbb{1}_{Y=i} \log p_i = - \log p_Y.$$

- **Účelová funkce**

- U **regrese** se akorát přidá *mean*, tedy: **mean squared error** a sečte se ztrátová funkce přes všechny body datasetu a zprůměruje se.

- U **binární/klasifikační úlohy** se akorát zahodí slovo *loss* a opět se pro každý bod sečte jeho ztrátová funkce a zprůměruje se.

Dopředný a zpětný chod

- Minimalizujeme účelovou funkci ($\mathbf{w}$) pomocí gradientního sestupu.
 - Při výpočtu gradientu používáme pravidlo pro **derivaci složené vícerozměrné funkce**.
 - Pro složení $\mathbf{y} = (y_1, \dots, y_N)^T, : \mathbb{R} \rightarrow \mathbb{R}$ s funkcí $f : \mathbb{R} \rightarrow \mathbb{R}$ se využívá **řetězové pravidlo**

$$x(\mathbf{x}) = \sum_{i=1}^N \left(\frac{\partial x}{\partial \mathbf{w}_i} \right) \mathbf{w}_i(\mathbf{x}).$$

- **Dopředný chod (forward pass)**
 - Výpočet výstupu sítě v bodě $\mathbf{x}$ se nazývá dopředný chod.
 - Dopředný chod provedeme pro všechny body trénovací množiny a spočítáme účelovou funkci.

- **Zpětný chod (backward pass)**
 - Pro výpočet gradientu účelové funkce ($\mathbf{w}$) podle $\mathbf{w}$ dostáváme -tý člen

$$w(\mathbf{w}) = 1/N \sum_{i=1}^N \left(\frac{\partial}{\partial \mathbf{w}_k} \right) (Y_i, (\mathbf{x}_i \mathbf{w}))_k (\mathbf{w}) w(\mathbf{x}_i \mathbf{w})$$

- Pro výpočet parciální derivace $\frac{\partial}{\partial \mathbf{w}_k} w(\mathbf{x}_i \mathbf{w})$ použijeme opět řetězové pravidlo a postupně bychom dostali

$$\begin{aligned} \frac{\partial}{\partial \mathbf{w}_k} w(\mathbf{x}_i \mathbf{w}) &= \\ \sum_{l=1}^{l-1} \left(\frac{\partial}{\partial \mathbf{w}_k} \right) \left(\frac{\partial}{\partial \mathbf{w}_l} \right) \left(\frac{\partial}{\partial \mathbf{w}_l} \right) \dots \left(\frac{\partial}{\partial \mathbf{w}_1} \right) (\mathbf{x}_i \mathbf{w}) &= \\ \sum_{l=1}^{l-1} \left(\frac{\partial}{\partial \mathbf{w}_k} \right) \left(\frac{\partial}{\partial \mathbf{w}_l} \right) \left(\frac{\partial}{\partial \mathbf{w}_l} \right) \dots \left(\frac{\partial}{\partial \mathbf{w}_1} \right) w(\mathbf{x}_i \mathbf{w}) &= \end{aligned}$$

- Toto řetězení aplikujeme dokud nenarazíme na vrstvu l a v ní neuron k , který přímo závisí na parametru $\mathbf{w}_k$. V tento moment zastavíme výpočet.
- Celkově pak pro l -tou složku gradientu účelové funkce dostáváme

$$w(\mathbf{w}) = 1/N \sum_{i=1}^N \left(\frac{\partial}{\partial \mathbf{w}_k} \right) (Y_i, (\mathbf{x}_i \mathbf{w})) \sum_{l=1}^{l-1} \left(\frac{\partial}{\partial \mathbf{w}_k} \right) \left(\frac{\partial}{\partial \mathbf{w}_l} \right) \dots \left(\frac{\partial}{\partial \mathbf{w}_1} \right) w(\mathbf{x}_i \mathbf{w}).$$

- Celý tento algoritmus nazýváme **zpětné šíření chyby** (angl. **back-propagation**).
- **Výpočetní graf**
 - Pro výpočet dopředného i zpětného chodu se obvykle používají kvůli efektivitě **výpočetní grafy**.

11. Vícevrstvé neuronové sítě - dávkové učení, optimalizační metody

Formulace optimalizační úlohy učení

- Chceme minimalizovat chybu predikce pomocí **účelové funkce**

$$J(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^N (Y_i - (\mathbf{x}_i \mathbf{w}))^2$$

vzhledem k parametrům $\mathbf{w}$.

- Pomocí **zpětného chodu** dostaneme $\nabla_{\mathbf{w}} J(\mathbf{w})$, který použijeme na update vah.
- Toto je **deterministická** metoda učení.
- U větších datasetů a velkého množství parametrů je ale výpočet zbytečně zdlouhavý a paměťově problematický.
- V jednom kroku se tedy využije jen **část trénovací množiny**.
- Gradient průměrné chyby na této části je pak **odhad** skutečného gradientu $\nabla_{\mathbf{w}} J(\mathbf{w})$.
- Této metodě se říká **dávkové učení**.

Dávkové učení

- V každém kroku je napočten gradient $\nabla_{\mathbf{w}} J(\mathbf{w})$ účelové funkce

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{b} \sum_{i=1}^b (Y_{(i)} - (\mathbf{x}_i \mathbf{w})) \mathbf{x}_i$$

kde velikost jedné dávky je b .

- Tento gradient se použije k updatu vah.
- Celý proces učení je tedy následující:
 - Náhodně promícháme body trénovací množiny.
 - Rozdělíme je na dávky o velikosti b .
 - Postupně **projdeme všechny dávky** a pro každou provedeme jeden krok gradientního sestupu.
 - Po N jsme prošli celou množinu.
 - Jednomu tomuto průchodu říkáme **epocha**.
 - Následně pokračujeme další epochou.
 - Na konci každé epochy **vyhodnotíme stav** natrénování a změříme výkonnost na validační množině.
 - Trénování ukončíme po dosažení určitého počtu epoch nebo pokud aktuální stav odpovídá požadovanému.

Optimalizační metody - SGD (s hybností), RMSProp, Adam

- **SGD (s hybností):**
 - Inicializujeme váhy $\mathbf{w}$ (a počáteční rychlost $\eta = 0$).
 - Zvolíme učicí parametr η (a parametr hybnosti α).
 - **Dokud nejsou splněny podmínky zastavení:**
 - Vezmeme dávku bodů z trénovací množiny.
 - Spočítáme gradient: $\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{b} \sum_{i=1}^b (Y_{(i)} - (\mathbf{x}_i \mathbf{w})) \mathbf{x}_i$.
 - (Spočteme rychlost: $\eta = \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$).

- Provedeme update vah:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} J(\mathbf{w}).$$

- *Nebo v případě s hybností:*

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}.$$

- **RMSProp:**

- Inicializujeme váhy $\mathbf{w}$, počáteční rychlost $\mathbf{v} = 0$ a akumulční proměnnou $\mathbf{s} = 0$.
- Zvolíme globální učicí parametr η , parametr hybnosti α , koeficient útlumu ϵ a malou konstantu δ .

- **Dokud nejsou splněny podmínky zastavení:**

- Vezmu dávku bodů z trénovací množiny.
- Spočítáme gradient: $\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{x}_i, y_i)$.
- Agregujeme kvadrát gradientu: $\mathbf{s} \leftarrow \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) \nabla_{\mathbf{w}} J(\mathbf{w}) + (1 - \alpha) \mathbf{s}$.
- Spočteme rychlost: $\Delta \mathbf{w} \leftarrow -\eta \nabla_{\mathbf{w}} J(\mathbf{w}) / \sqrt{\mathbf{s} + \delta}$.
- Provedeme update vah: $\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$.

- **ADAM (ADAPtive Moments)**

- Inicializujeme váhy $\mathbf{w}$, akumulční proměnné $\mathbf{m} = 0$, $\mathbf{v} = 0$ a časový krok $t = 0$.
- Zvolíme globální učicí parametr η , parametr hybnosti α , koeficienty útlumu β_1, β_2 a malou konstantu ϵ .

- **Dokud nejsou splněny podmínky zastavení:**

- Vezmeme dávku bodů z trénovací množiny.
- Spočteme gradient.
- Zvedneme časový krok: $t \leftarrow t + 1$.
- Agregujeme první moment: $\mathbf{m} \leftarrow \beta_1 \nabla_{\mathbf{w}} J(\mathbf{w}) + (1 - \beta_1) \nabla_{\mathbf{w}} J(\mathbf{w})$.
- Agregujeme druhý moment: $\mathbf{v} \leftarrow \beta_2 \nabla_{\mathbf{w}} J(\mathbf{w}) \nabla_{\mathbf{w}} J(\mathbf{w}) + (1 - \beta_2) \nabla_{\mathbf{w}} J(\mathbf{w}) \nabla_{\mathbf{w}} J(\mathbf{w})$.
- Provedeme korekci obou momentů: $\hat{\mathbf{m}} \leftarrow \mathbf{m} / (1 - \beta_1^t)$, $\hat{\mathbf{v}} \leftarrow \mathbf{v} / (1 - \beta_2^t)$.
- Provedeme update vah: $\mathbf{w} \leftarrow \mathbf{w} - \eta \hat{\mathbf{m}} / \sqrt{\hat{\mathbf{v}} + \epsilon}$.

12. Vícevrstvé neuronové sítě - hluboké učení, regularizace

Problémy trénování hlubokých sítí

- Gradientní sestup mívá do směru nulového gradientu, což nemusí být globální minimum.
- Problém je, že u sítí s velkým počtem neuronů může snadno dojít k **přeučení**.
- Malý dataset se neuronka dokáže **naučit nazpaměť**.
- Používáme tedy různé druhy **regularizace**.

Metody regularizace - penalizace norem parametrů, předčasné zastavení, dropout, dávková normalizace

- **Penalizace norem parametrů**

- K účelové funkci $J(\mathbf{w})$ přidáme penalizační člen $\lambda \|\mathbf{w}\|^2$.

- Nová účelová funkce je tedy

$$J(\mathbf{w}) = J(\mathbf{w}) + \lambda J(\mathbf{w}),$$

kde λ je hyperparametr kontrolující sílu regularizace.

- Penalizace se přidává po vrstvách.
- Při ℓ_2 regularizaci pro i -tou vrstvu se přidá člen

$$J(\mathbf{w}) = \|\mathbf{w}^{(i)}\|_2^2$$

- Při ℓ_1 regularizaci pro i -tou vrstvu se přidá člen

$$J(\mathbf{w}) = \|\mathbf{w}^{(i)}\|_1$$

- Regularizační efekt podobný jako u **Ridge** nebo **Lasso**.

- **Předčasné zastavení (early stopping)**

- Na konci každé epochy se spočte validační chyba a ukládá se ta nejlepší.
- Pokud se po l epochách nezmenší nejlepší validační chyba, ukončíme učení a vrátíme váhy odpovídající nejmenší validační chybě.

- **Dropout**

- Vynulování vybraných **vstupů do vrstvy**, na kterou je aplikován.
- Nulování jenom v trénovací fázi, **pro každou dávku nezávisle**.
- Každý vstup je nulován s pravděpodobností $p \in (0, 1)$.
- Nutí model soustředit se na různé aspekty a nesoustředit se pořád na to samé.
- Značně zvyšuje schopnost **generalizovat**.

- **Inverze dropout**

- Nevynulované vstupy jsou ještě přeškálovány číslem $1/(1-p)$.

- **Dávková normalizace**

- Pro hlubší sítě jsou při gradientním sestupu **parametry v mělčích vrstvách ovlivňovány parametry z hlubších vrstev**.
- Proti tomuhle slouží dávková normalizace.
- Pro danou dávku dat výstupy z určené skryté vrstvy **standardizujeme** a poté (volitelně) **lineárně ztransformujeme**.
- Zbavíme se **komplikované interakce parametrů v hlubších vrstvách**.

13. Konvoluční neuronové sítě

Operace konvoluce a její vlastnosti

- **Definice konvoluce:**

Nechť f a g jsou dvě funkce reálné proměnné, absolutně integrovatelné na celém $\mathbb{R}$.

Konvoluce funkcí f a g je definována vztahem

$$(\cdot)(\cdot) = (\cdot)(-\cdot)$$

pro každé $\in \mathbb{R}$.

- **Vlastnosti:**

- Komutativita: $=$
- Vztah vůči derivaci: $(\cdot)(\cdot) = (\cdot)(\cdot)$
- Ekvivariance vůči translaci: $(\cdot)(a + \cdot) = ((x + a) \cdot)(\cdot)$

- **Definice diskrétní konvoluce:**

Nechť $(x(i))_{i=-\infty}^{+\infty}$ a $(w(i))_{i=-\infty}^{+\infty}$ jsou dvě posloupnosti.

Diskrétní konvoluci posloupností x a w definujeme vztahem

$$(x \cdot w)(i) = \sum_{j=-\infty}^{+\infty} x(j)w(i-j)$$

pro každé $i \in \mathbb{Z}$.

- **Vlastnosti:**

- Analogické s konvolucí pro funkce.

- **Konvoluce v neuronových sítích**

- Uvažujme 2D obrázek o rozměrech $p_1 \times p_2$, kdy použijeme dvourozměrné jádro o rozměrech $k_1 \times k_2$ a spočteme

$$(i, j) = (\cdot)(i, j) = (\cdot)(i - k_1 + 1, j - k_2 + 1)$$

- Z komutativity

$$(i, j) = (\cdot)(i, j) = (i - k_1 + 1, j - k_2 + 1)(\cdot)$$

- A jelikož se učíme složky, uděláme

$$(i + k_1 - 1, j + k_2 - 1)(-k_1 + 1, -k_2 + 1)$$

- Tomuto se říká **křížová korelace**

$$(\cdot)(i, j) = \sum_{k_1=1}^{k_1} \sum_{k_2=1}^{k_2} (i + k_1 - 1, j + k_2 - 1)(\cdot)$$

kde $1 \leq i \leq p_1 - k_1 + 1$ a $1 \leq j \leq p_2 - k_2 + 1$.

Konvoluční vrstva

- Aplikujeme na n -rozměrné tenzory s c kanály.
- V **channel-first** notaci jednomu datovému bodu odpovídá tenzor o rozměrech $(1, p_1, \dots, p_n)$.
- Jeden výstupní kanál konvoluční vrstvy s jádrem o rozměrech $(k_1, \dots, k_n)$ má pro každý vstupní kanál jeden tenzor o těchto rozměrech a výsledné konvoluce (křížové korelace) se přes kanály sčítají.
- K tomuto se přičte ještě intercept b .
- Obecně na výstupu máme c_o kanálů a i -tý kanál výstupního tenzoru o rozměrech $(1, p_1 - k_1 + 1, \dots, p_n - k_n + 1)$ je určen vztahem

$$= b + \sum_{l=1}^n w_l \cdot l.$$

- Složky jádra i intercept jsou parametry, které se trénují gradientním sestupem.

Porovnání s plně propojenými vrstvami - princip omezeného spojení a sdílení parametrů

- Konvoluční vrstva **sníží počet parametrů** s klasickými plně propojenými vrstvami.
- Každý výstupní bod je propojen pouze s omezenou oblastí vstupu (**oblast vnímání**).
- Tomuto se říká **princip omezeného spojení**.
- **Sdílení parametrů**
 - Konvoluční vrstvu můžeme chápat jako **plně propojenou vrstvu s apriorně svázanými a lokálně omezenými parametry**.
 - **Apriorně svázané** = Apriorně (předem) se určí, že se pro různé části dat použijí naprosto stejné váhy. Vezmu malý filtr a tím projedu celý obrázek.
 - **Lokálně omezené** = Neuron se nekouká na všechna data, ale jen na malý výřez ve svém okolí.

Pooling, padding, stride

- **Pooling**
 - Konvoluční vrstva se typicky realizuje pomocí **konvoluce, aplikace nelineární aktivační funkce a sdružovací funkce**.
 - Funguje to jako taková **komprese**.
 - Vezmu čtvereček (třeba 2x2) a smrsknu ho do jednoho čísla.
 - Nejtypičtějším příkladem sdružovací funkce je **max pooling**, kde vezmu pixel s největší hodnotou.
 - Můžu použít i **average pooling** a podobné.
- **Padding**
 - Před výpočtem se rozšíří vstupní tenzor ve všech směrech o daný počet hodnot.
 - Typicky doplňuji nuly nebo hodnoty z okraje, či zrcadlení.
- **Stride**
 - Normálně posouváme indexy konvoluce nebo paddingu o jedna.
 - Tento **skok** se nazývá **stride** a můžeme skákat i o víc než jedna.
 - Ekvivalentní (ne však výpočetně) jako provedení původního výpočtu a vyházení části vypočítaných hodnot.

14. Strojové zpracování přirozeného jazyka (nebude zkoušeno)

Maticová reprezentace textů v bag of words modelu

Stop slova, lemmatizace, Tf-idf

15. Posilované učení

Formulace problému a základních pojmů posilovaného učení

- Uvažujeme systém, kde můžeme vykonávat **akce** a snažíme se dosáhnout nějakého **cíle**.
- Tomuto systému říkáme **agent**.
- **Posilované učení** je snaha naučit agenta se chovat tak, aby **maximalizovat** nějakou **odměnu**.
- Neříkáme, jaké přímo akce má agent vykonávat, ale agent musí zkoušením zjistit, co je nejvýhodnější.
- Snaží se maximalizovat **všechny budoucí odměny**.
- Agent se nachází v **prostředí**, které je v nějakém **stavu** a na základě toho se agent rozhodne, jakou akci vykoná.
- Typicky obdrží po vykonání akce **okamžitou odměnu**.
- **Příklady:**
 - Piškvorky
 - Šachy
 - Go
 - apod.

Explorace versus exploitace

Agent se neustále snaží balancovat mezi dvěma přístupy.

- **Explorace**
 - Agent zkouší různé možnosti a snaží se zjistit, jaké akce jsou pro něj nejvýhodnější.
- **Exploitace**
 - Agent na základě znalostí z explorace opakuje tu akci, která mu přináší největší odměnu.
- **Problém explorace vs exploitace:**
 - Když agent bude trávit příliš mnoho času v **explorační** fázi, bude **zbytečně trávit čas neoptimálními kroky a přijde o možnou odměnu**.
 - Když ale zase příliš brzo vkročíme do fáze **exploitace**, můžeme minout lepší řešení a **exploitování** tohoto **horšího řešení nepovede k maximalizaci odměny**.

k -ruký bandita a základní algoritmy jeho řešení

- **k -ruký bandita:** Máme k dispozici několik akcí, každá s jinou distribucí odměn.
- A : **Akce** zvolená v čase t .
- R_t : **Odměna** obdržená v čase t po vykonání akce.
- $v_t(a)$: Očekávaná odměna při zvolené akci a , nazýváme **hodnota akce a** . Definujeme jako $v_t(a) = \mathbb{E} R_t | A = a$.

- Funkce $(a) : \rightarrow \mathbb{R}$ se nazývá **hodnotová funkce**.
- Kdybychom znali (a) všech akcí, tak optimální strategie bude vždy volit $g_a(a)$.
- To ale neznáme a tak v průběhu vytváříme **odhad**.
- $(a) : \mathbf{Odhad hodnoty akce } a \text{ v čase } .$
- Nejpřirozenější způsob sestrojení odhadu je použití výběrového průměru odměn

$$(a) = \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{A=a} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{A=a}.$$

- Ze zákona velkých čísel tento odhad konverguje ke střední hodnotě (a) .

• Algoritmy

• greedy search

- Vybereme akci A jako

$$A = \arg \max_a (a).$$

- Pokud je nejvyšší pro několik akcí, vybereme náhodně.

• Nevýhody:

- Nevhodné, když naše odhady (a) nejsou moc dobré.
- Pokud na začátku dostaneme špatnou odměnu, už se z toho nevyhrabeme.
- Nevhodné pro nestacionární systém (mění se v čase).

• -greedy search

- Inicializujeme pro všechna $a : (a) = 0$ a $N(a) = 0$.

- Zvolíme parametr ϵ .

• Opakujeme:

- $A = \arg \max_a (a) - \epsilon \sqrt{\frac{\ln N(A)}{N(A)}}$
- $N(A) = N(A) + 1$
- $(A) = (A) + \frac{1}{N(A)} (A) - \frac{1}{N(A)} (A)$

• Výhody:

- V limitě najde optimum.

• Nevýhody:

- Po nalezení optima stále bude zkoušet s ϵ nevýhodné akce.
- ϵ je parametr, který musíme ladit.

• Optimistický greedy search

- Zavedeme apriorní znalost systému.
- Dosáhneme tak zlepšení explorační.
- K tomu stačí nastavit (a) na stejné, vysoké hodnoty.

• Výhody:

- Silně a systematicky podporuje explorační na začátku.
- Mnohokrát konverguje rychleji než -greedy.

- **Nevýhody:**

- V nestacionárním systému selhává, protože prvotní optimismus vyprchá a pak je to čistý greedy search.
- Vyžaduje expertní znalost systému (maximální možná odměna) k nastavení počáteční hodnoty.

- **UCB (Upper Confidence Bound):**

- Zavádí **systematickou exploraci** (místo náhodné pomoci).
- A volíme jako:

$$A = g_a(a) + \sqrt{\ln N(a)},$$

kde je parametr kontrolující míru explorační.

- Kombinuje se snaha **vybrat aktuálně nejlepší stav** se snahou **redukovat nejistotu**.

- **Výhody:**

- Cíleně vybírá akci, která je buď velmi slibná nebo málo prozkoumaná.

- **Nevýhody:**

- Problémy v nestacionárním případě.

- *Markovský rozhodovací problém: Nezohledňují historii a řídí se pouze současným stavem a prostředím.*